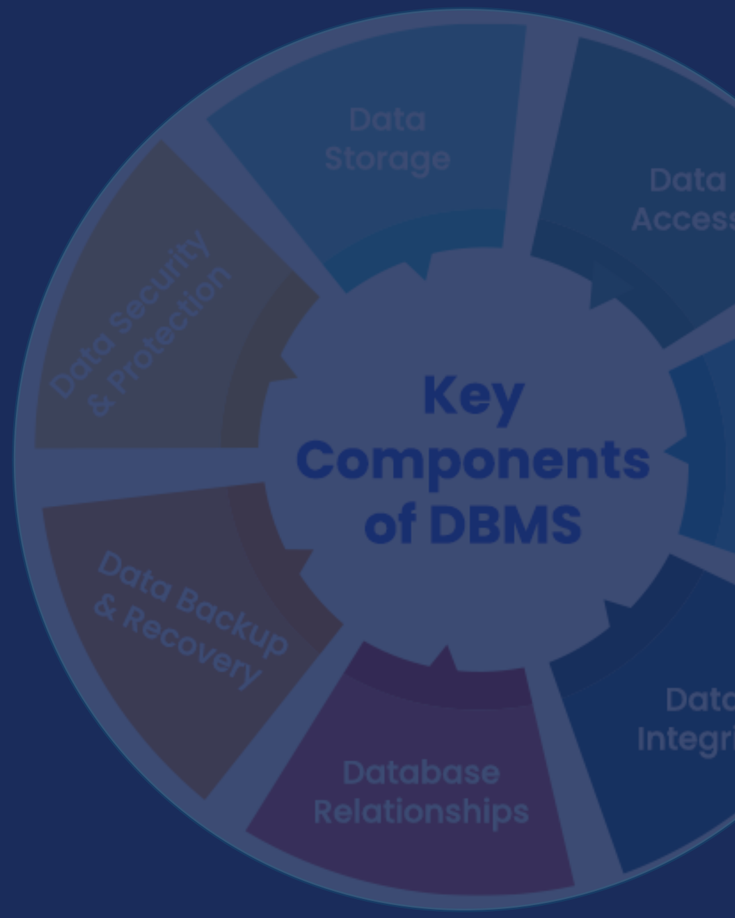
# Secure Placement Data Tracking & Analysis System

*A DBMS-Powered Placement Management System*

**Presented by:** **Mahi Sirohi (RA2411030030053)**
**Arpita Singh Baghel(RA2411030030048)**



# Introduction

OVERVIEW

## What Is It?

- A web-based system to manage student placement activities
- Tracks students, companies, job openings & applications
- Stores all data in a structured MySQL database
- Replaces manual spreadsheets with an organized digital system
- Easy to use for both admins and placement coordinators

## Why Is It Needed?

- Managing placement data manually is error-prone & slow
- Difficult to track multiple students across many companies
- No easy way to search or filter placement records
- Hard to generate reports from scattered spreadsheets
- A database-driven system solves all these problems

# Objectives

GOALS

1

### Build a centralized database

Store all placement-related data in one organized MySQL database

2

### Manage student profiles

Add, update, view and delete student records easily

3

### Track job applications

Know which student applied to which company and what the result was

4

### Handle company & job data

Store company info and available job openings with required skills

5

### Perform full CRUD operations

Create, Read, Update, Delete records using Flask + MySQL

6

### Demonstrate DBMS concepts

Apply primary keys, foreign keys, joins, and normalization in practice

# Technologies Used

TECH STACK

## Frontend

- HTML5 – Page structure & forms
- CSS3 – Styling and layout
- JavaScript – Client-side interactivity

## Backend

- Python 3 – Core programming language
- Flask – Lightweight web framework
- Jinja2 – HTML templating engine

## Database

- MySQL – Relational database
- MySQL Connector – Python–DB link
- SQL – Queries for all operations

# System Architecture

FLOW

| User (Browser) | Frontend (HTML/CSS/JS) | Flask Backend | MySQL Database | Response (JSON/HTML) |
|---|---|---|---|---|
| Student / Admin opens the web app and interacts via browser | Forms and tables collect input and display results | Routes process requests, run business logic, talk to DB | Stores all records; returns query results to Flask | Data sent back and displayed to the user in real time |

# Database Design

SCHEMA

## TABLE: STUDENTS

- student_id (PK)
- name
- email
- branch
- cgpa

## TABLE: COMPANIES

- company_id (PK)
- company_name
- industry
- location

## TABLE: JOBS

- job_id (PK)
- company_id (FK)
- role
- package
- criteria

## TABLE: APPLICATIONS

- app_id (PK)
- student_id (FK)
- job_id (FK)
- status
- date

**PK** = Primary Key | **FK** = Foreign Key (links tables together)

# Frontend – User Interface

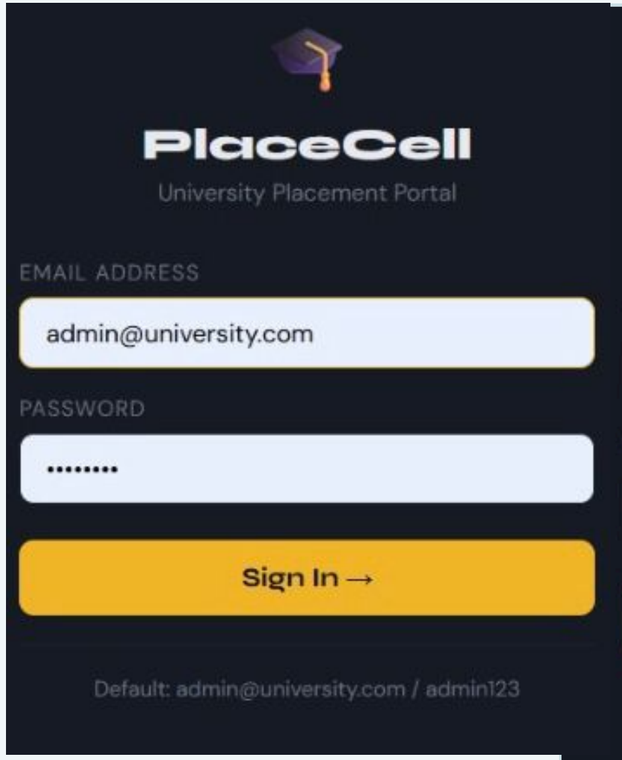
UI/UX

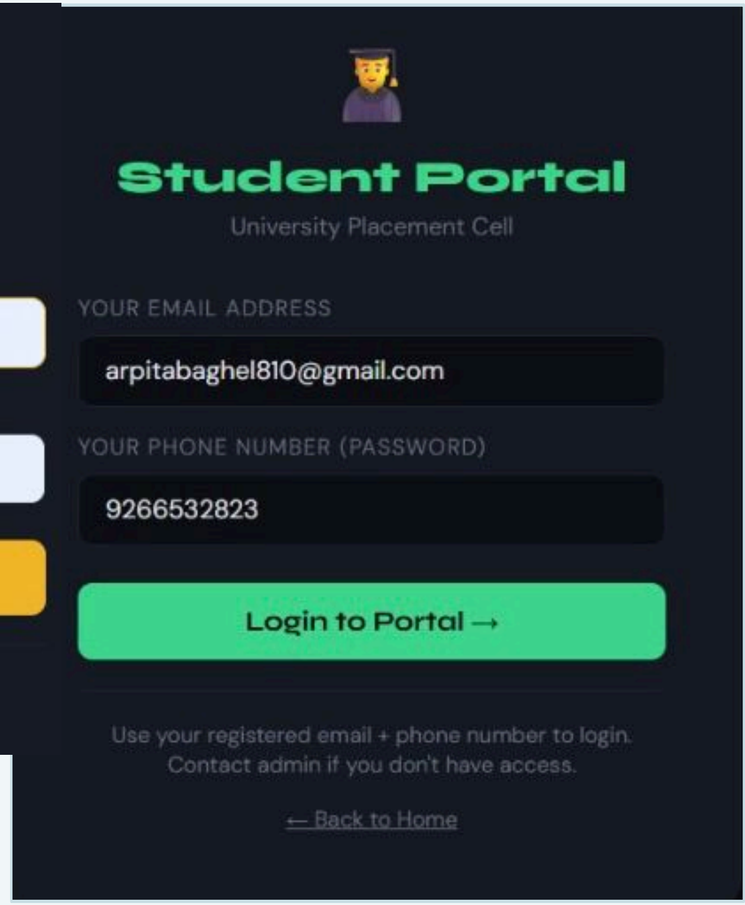
## Pages & Forms

- Login Page – Admin sign-in form
- Dashboard – Summary of all placements
- Student Form – Add or edit student info
- Company/Job Form – Manage company listings
- Application Form – Submit and track applications
- Records Table – View all entries at a glance

## UI Features

- Responsive layout – works on any screen
- Form validation – prevents empty submissions
- Dynamic tables – display records from database
- Dropdown menus – link related tables (FK)
- Search/filter – find students or companies quickly
- Flash messages – success and error notifications



# Backend – Flask Framework

PYTHON FLASK

### Flask Routing

Each URL maps to a Python function
that handles the request logic

### DB Connection

Flask connects to MySQL using mysql-connector-python library

### CRUD Operations

Routes perform INSERT, SELECT, UPDATE and DELETE using SQL

### Jinja2 Templates

HTML pages receive data from Flask and render it dynamically

**Flask Route Example (app.py)**

```
@app.route('/add_student', methods=['GET', 'POST'])
def add_student():
    if request.method == 'POST':
        name = request.form['name']   # Get form data
        cursor.execute("INSERT INTO students (name) VALUES (%s)", (name,))
        return redirect('/students')   # Go back to list
```

# API Concept – Flask Routes as Endpoints

API / ROUTES

## What is an API?

- API = Application Programming Interface
- It is a bridge between frontend and backend
- Flask routes work as simple API endpoints
- Browser sends a request → Flask responds with data

## GET Method

- Used to fetch / read data from the server
- Example: Open /students → shows student list
- Does not change any data in the database
- Safe to call multiple times without side effects

## POST Method

- Used to send / submit data to the server
- Example: Submit form → inserts new student
- Data travels in the request body (not URL)
- Changes the database (INSERT or UPDATE)

**Example Routes:** **GET /students** → Show all students **POST /add_student** → Add a new student

# Database Operations – SQL Queries

CRUD SQL

## INSERT

Add new record

```
INSERT INTO students (name, branch, cgpa)
VALUES ('Rahul', 'CSE', 8.5);
```

## SELECT

Read / fetch records

```
SELECT s.name, j.role FROM students s
JOIN applications a ON s.student_id =
a.student_id;
```

## UPDATE

Modify existing record

```
UPDATE applications
SET status = 'Hired'
WHERE app_id = 5;
```

## DELETE

Remove a record

```
DELETE FROM applications
WHERE app_id = 5;
```

# Key Features of the System

FEATURES

### Student Management

Add, view, edit & delete student records with full profile info

### Company Management

Store company profiles and link them to job postings

### Job Listings

Manage job roles, package details, and eligibility criteria

### Application Tracking

Track which student applied where — and the current status

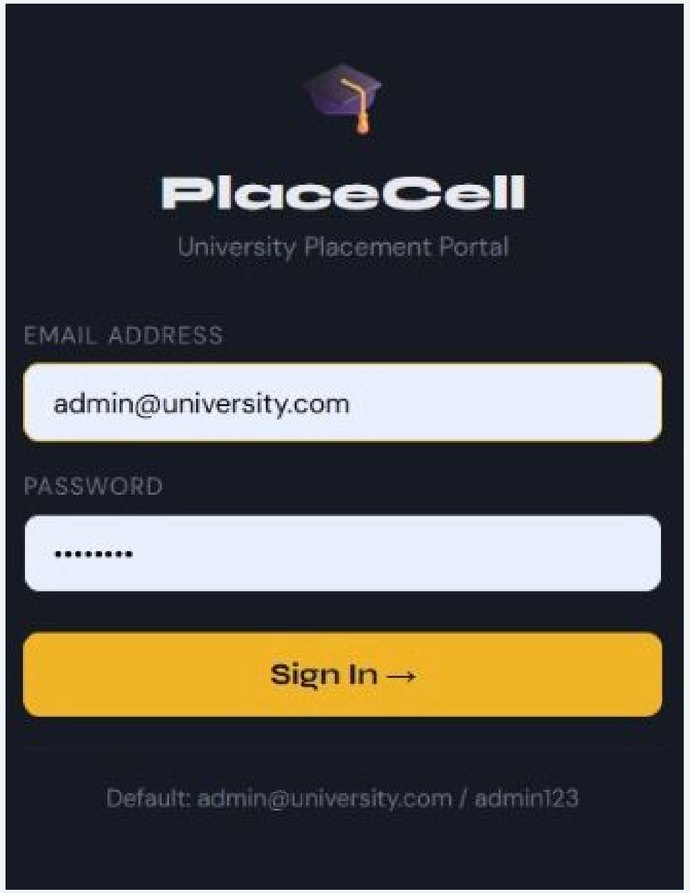
### Search & Filter

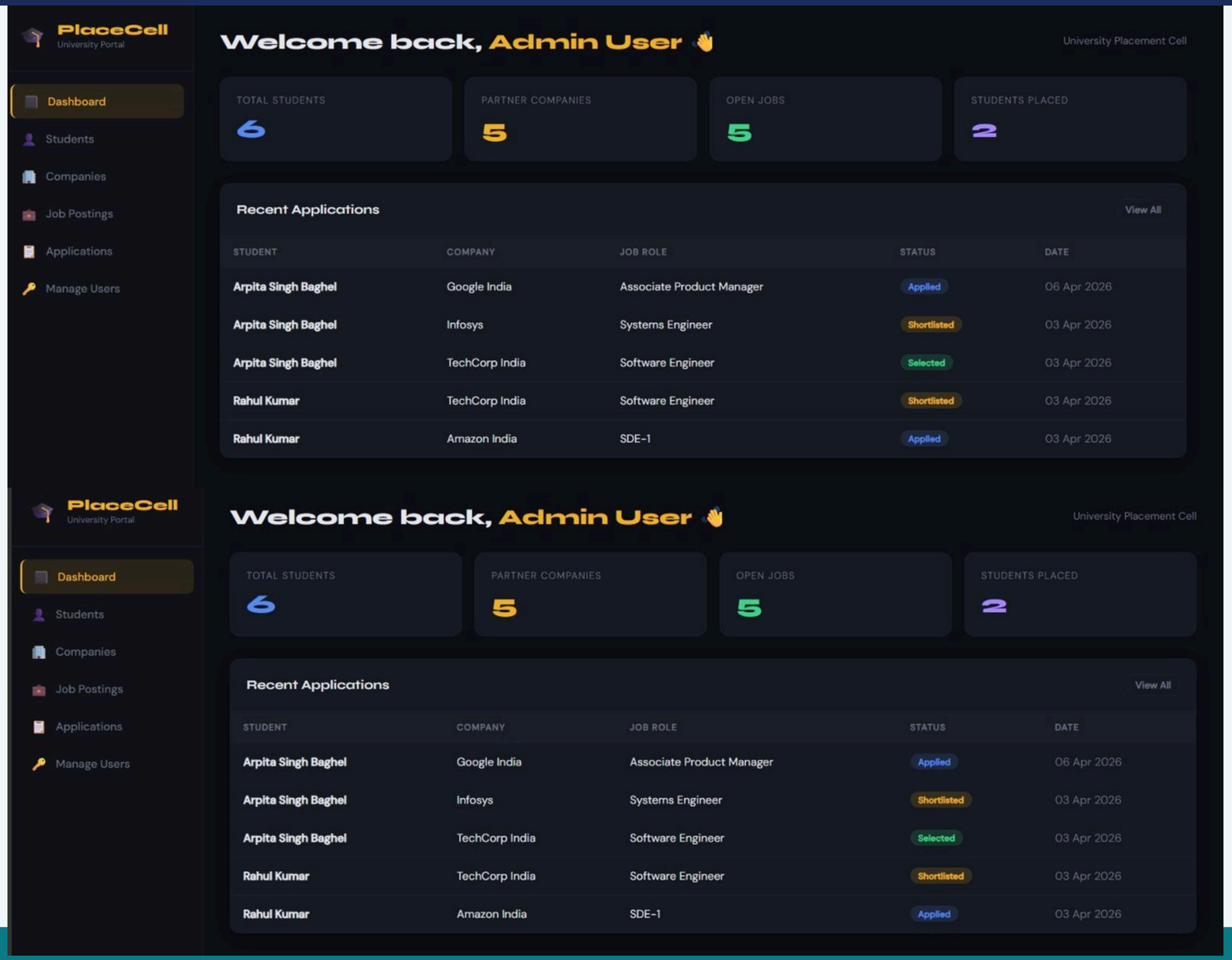
Quickly find students or companies from large datasets

### Dashboard Overview

Summary stats: total students, placements, companies, etc.

# Output / System Screenshots




PlaceCell
University Portal
Welcome back, Admin User 👋
University Placement Cell
Dashboard
Students
Companies
Job Postings
Applications
Manage Users
TOTAL STUDENTS
6
PARTNER COMPANIES
5
OPEN JOBS
5
STUDENTS PLACED
2
Recent Applications
View All
STUDENT
COMPANY
JOB ROLE
STATUS
DATE
Arpita Singh Baghel
Google India
Associate Product Manager
Applied
06 Apr 2026
Arpita Singh Baghel
Infosys
Systems Engineer
Shortlisted
03 Apr 2026
Arpita Singh Baghel
TechCorp India
Software Engineer
Selected
03 Apr 2026
Rahul Kumar
TechCorp India
Software Engineer
Shortlisted
03 Apr 2026
Rahul Kumar
Amazon India
SDE-1
Applied
03 Apr 2026

# Conclusion

- We successfully built a full-stack Student Placement Tracker using HTML, CSS, JS, Python Flask, and MySQL
- The system applies core DBMS concepts: tables, primary keys, foreign keys, relationships, and SQL queries
- CRUD operations are fully functional — data can be added, viewed, updated, and deleted seamlessly
- The project demonstrates how a real-world web application connects a user interface to a live database
- This system can directly replace manual placement tracking in a college environment

*"A well-designed database is the backbone of every great application."*

# Future Scope

ENHANCEMENTS

01

### Student Login Portal

Allow individual students to log in, view their applications, and receive email notifications

02

### Analytics Dashboard

Show placement statistics — charts for branch-wise, year-wise, and package-wise data

03

### REST API with JSON

Convert Flask routes into a proper REST API so mobile apps can also use the backend

04

### Resume Upload

Enable PDF resume uploads and link them to student profiles in the database

05

### Cloud Deployment

Deploy the application to AWS or Heroku so it is accessible from anywhere online

06

### AI Matching

Suggest best-fit students for a job based on their CGPA, skills, and branch using ML